

BLOG MANAGEMENT SYSTEM

Full Stack Development Internship Project Report

Submitted By

Kusum Chaurasia

B.Tech (Computer Science & Engineering)

Dronacharya Group of Institutions, Greater Noida

Submitted To

Codomax Digital Solutions

Internship

Full Stack Development Internship

Project Duration

July 2026 – August 2026

Submission Date

August 2026

Certificate

This is to certify that the project entitled "**Blog Management System**" has been successfully completed by **Kusum Chaurasia** as part of the **Full Stack Development Internship** conducted by **Codomax Digital Solutions**.

The project was developed using modern web development technologies including **HTML, CSS, JavaScript, Node.js, Express.js, Git, and GitHub**. The work presented in this report is original and has been completed under the internship guidelines. During the development process, the project was tested, improved, and documented according to the internship requirements.

This documentation represents the practical knowledge and technical skills acquired throughout the internship.

Acknowledgement

I would like to express my sincere gratitude to **Codomax Digital Solutions** for providing me with the opportunity to participate in the Full Stack Development Internship. This internship helped me gain practical experience in developing modern web applications and understanding the workflow of real-world software development.

I would also like to thank the mentors and trainers who guided me throughout the internship by providing valuable learning resources, daily tasks, and continuous support. Their guidance helped me improve my technical skills and understand the concepts of frontend development, backend development, REST APIs, and version control using Git and GitHub.

Finally, I would like to thank my college and everyone who encouraged and supported me during the successful completion of this project.

Table of Contents

1. Introduction
2. Objectives
3. Project Description
4. Technologies Used
5. Software Requirements
6. System Requirements
7. Project Features
8. Folder Structure
9. Project Workflow
10. API Details
11. Project Screenshots
12. Challenges Faced
13. Future Scope
14. Conclusion
15. References

1. Introduction

The **Blog Management System** is a Full Stack Web Development project designed to provide users with an easy and efficient platform to manage blog posts. The project allows users to create, edit, view, and delete blogs through a simple and user-friendly interface. It demonstrates how frontend and backend technologies work together to build a complete web application.

The frontend of the project has been developed using **HTML, CSS, and JavaScript**, which provide an attractive and responsive user interface. The backend has been developed using **Node.js and Express.js**, which handle API requests and manage blog data. The project uses the Fetch API to establish communication between the frontend and backend.

The primary objective of this project is to understand the complete development process of a Full Stack Web Application, including user interface design, form validation, backend development, API integration, and project deployment. The project also demonstrates the use of Git and GitHub for version control and collaboration.

The Blog Management System is a beginner-friendly project that provides hands-on experience in implementing CRUD (Create, Read, Update, Delete) operations. Throughout this internship project, several additional features such as responsive design, social media integration, contact forms, animations, and GitHub Pages deployment were also implemented to improve the overall user experience.

This project serves as an excellent learning platform for understanding modern web development concepts and industry-standard development practices.

2. Objectives

The main objective of developing the Blog Management System is to understand the complete lifecycle of Full Stack Web Development by building a real-world project.

The specific objectives of this project are:

- To develop a responsive and user-friendly Blog Management System.
- To understand the fundamentals of frontend and backend integration.
- To perform CRUD (Create, Read, Update, Delete) operations using REST APIs.
- To implement client-side form validation using JavaScript.
- To develop backend APIs using Express.js.
- To establish communication between frontend and backend using the Fetch API.
- To improve coding standards through clean and organized project structure.
- To practice version control using Git and GitHub.
- To deploy the frontend using GitHub Pages.
- To enhance problem-solving and debugging skills during project development.
- To gain practical experience in software development as part of the Full Stack Development Internship.

3. Project Description

The **Blog Management System** is a Full Stack Web Development project that provides an easy and efficient platform for creating and managing blogs. The main purpose of this project is to help users perform all basic blog management operations through a simple and responsive interface.

The project consists of multiple webpages, including **Home**, **Add Blog**, **Categories**, **About**, and **Contact**. The Home page displays the latest blog posts, while the Add Blog page allows users to create new blogs by filling out a form. Each blog contains information such as title, author name, category, summary, content, and other details.

The project also supports **Edit** and **Delete** operations, allowing users to modify or remove existing blog posts. These operations are performed through REST APIs developed using **Express.js**. The frontend communicates with the backend using the **Fetch API**, enabling dynamic data exchange without reloading the webpage.

To improve the user experience, form validation has been implemented using JavaScript. Responsive design techniques ensure that the website works properly on desktop, tablet, and mobile devices. Additional features such as social media links, contact information, and smooth navigation make the application more user-friendly.

Overall, this project demonstrates the complete workflow of a Full Stack Web Application and provides practical experience in frontend development, backend development, API integration, and project deployment.

4. Technologies Used

The Blog Management System has been developed using modern web development technologies. Each technology plays an important role in the development process.

HTML5

HTML5 is used to create the structure of all webpages in the project. It defines headings, paragraphs, images, forms, tables, navigation bars, buttons, and other page elements. Every webpage in the project is built using semantic HTML tags, making the website well-structured and easy to understand.

CSS3

CSS3 is used to design the user interface of the project. It provides colors, fonts, spacing, layouts, animations, hover effects, and responsive design. CSS helps create an attractive and professional appearance while ensuring the website works properly on different screen sizes.

JavaScript

JavaScript adds interactivity to the project. It is used for form validation, button click events, dynamic content loading, Fetch API requests, editing blog posts, deleting blog posts, and improving the overall user experience. JavaScript makes the webpages interactive instead of static.

Node.js

Node.js provides the runtime environment for executing JavaScript on the server side. It helps create the backend server that processes requests from the frontend and returns appropriate responses.

Express.js

Express.js is a lightweight web framework built on Node.js. It is used to develop REST APIs for creating, displaying, editing, and deleting blog posts. Express.js simplifies routing and request handling.

Git

Git is a version control system that tracks project changes. It helps maintain project history and allows developers to manage different versions of the project efficiently.

GitHub

GitHub is used to store the project online. It allows project sharing, version management, collaboration, and project deployment using GitHub Pages.

GitHub Pages

GitHub Pages is used to host the frontend of the project online. It allows anyone to access the website using a public URL without installing the project locally.

Visual Studio Code

Visual Studio Code is the primary code editor used during project development. It provides useful extensions, debugging tools, syntax highlighting, and Git integration.

Postman

Postman is used to test REST APIs developed in the backend. It helps verify whether GET, POST, PUT, and DELETE requests are working correctly before integrating them with the frontend.

5. Software Requirements

The following software is required to develop and run the Blog Management System successfully:

- Operating System: Windows 10 or Windows 11
- Code Editor: Visual Studio Code
- Runtime Environment: Node.js
- Package Manager: npm (Node Package Manager)
- Web Browser: Google Chrome
- Version Control System: Git
- Online Repository: GitHub
- API Testing Tool: Postman
- Live Server Extension for Visual Studio Code

All these software tools work together to create, test, and deploy the project efficiently.

6. System Requirements

The minimum hardware requirements for running this project are:

- Processor: Intel Core i3 or above
- RAM: Minimum 4 GB
- Storage: At least 2 GB free disk space
- Internet Connection: Required for GitHub and package installation
- Display Resolution: 1366 × 768 or higher

Recommended hardware requirements include:

- Intel Core i5 Processor or above
- 8 GB RAM
- SSD Storage
- Stable Internet Connection

These specifications provide better performance during development and testing.

7. Project Features

The Blog Management System includes several important features that improve usability and functionality.

Home Page

The Home page displays the latest blog posts in an organized layout. Users can quickly browse available blogs and access different sections through the navigation menu.

Add Blog

The Add Blog page provides a form where users can enter blog details such as title, author, category, summary, content, and other information. JavaScript validation ensures that only valid data is submitted.

View Blogs

Users can view all available blogs directly on the Home page. Blog data is loaded dynamically through backend APIs.

Edit Blog

The Edit Blog feature allows users to update existing blog information without creating a new blog entry.

Delete Blog

Users can permanently remove unwanted blog posts through the Delete button after confirmation.

Categories

The Categories page organizes blogs into different categories, making it easier for users to browse content.

About Page

The About page provides information about the Blog Management System, its purpose, objectives, and development details.

Contact Page

The Contact page allows users to send messages and access contact information. It also includes social media links for better communication.

Responsive Design

The website adjusts automatically according to different screen sizes, providing a consistent experience on desktop, tablet, and mobile devices.

Form Validation

JavaScript validation ensures that users provide valid information before submitting forms, reducing incorrect data entry.

REST API Integration

The frontend communicates with the backend using REST APIs developed in Express.js, allowing dynamic creation, retrieval, updating, and deletion of blog posts.

Perfect! Chalo **Part 3** banate hain. Ye documentation ka sabse important part hai. Isme **System Architecture, Folder Structure Explanation, Project Workflow aur API Details** ko detail me explain kiya gaya hai.

8. System Architecture

The Blog Management System follows a simple **three-layer architecture**, which consists of the **Frontend**, **Backend**, and **Data Layer**. Each layer has a specific responsibility and communicates with the others to provide complete functionality.

Frontend Layer

The Frontend is the user interface of the application. It is developed using **HTML5, CSS3, and JavaScript**. This layer allows users to interact with the system through different webpages such as Home, Add Blog, Categories, About, and Contact.

The frontend collects user input through forms and sends requests to the backend using the **Fetch API**. It also receives responses from the backend and displays blog information dynamically on the webpages.

Backend Layer

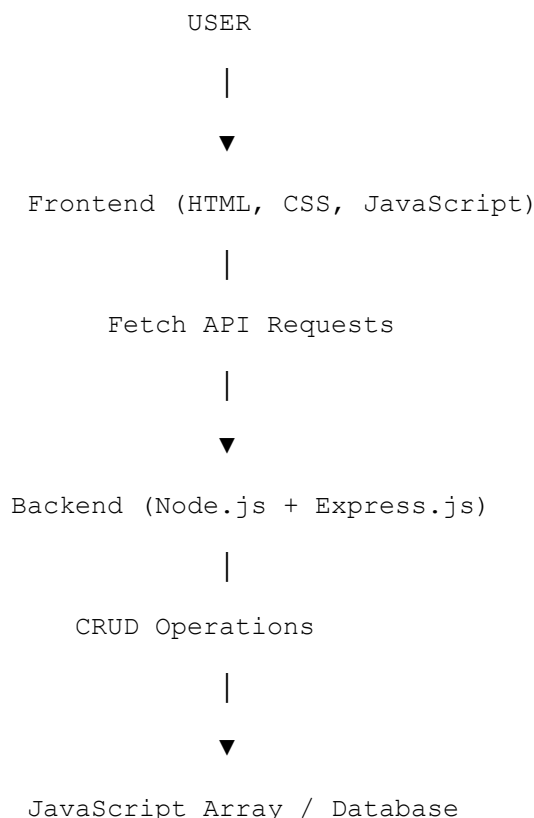
The Backend is developed using **Node.js** and **Express.js**. It handles all API requests sent from the frontend. The backend is responsible for creating new blogs, displaying existing blogs, updating blog details, and deleting blogs.

It processes user requests, performs the required operations, and sends appropriate responses back to the frontend.

Data Layer

The current version of the project stores blog data temporarily using a JavaScript array. During project execution, the backend performs CRUD operations on this data. In future versions, this layer can be replaced with a database such as **MongoDB** or **MySQL** for permanent storage.

Architecture Diagram



9. Folder Structure Explanation

The project follows an organized folder structure that makes the code easier to understand and maintain.

index.html

This is the main homepage of the website. It displays featured blogs, latest blogs, categories, newsletter, and footer information.

add-blog.html

This page contains the blog submission form where users can enter all blog details and create new blog posts.

categories.html

This page displays different blog categories and helps users browse blogs based on topics.

about.html

The About page explains the purpose of the project and provides information about the Blog Management System.

contact.html

The Contact page contains the contact form, contact information, office location, and social media links.

style.css

This file contains all styling rules used throughout the website, including colors, layouts, buttons, forms, animations, and responsive design.

script.js

This JavaScript file handles form validation, blog submission, edit functionality, and user interactions.

home.js

This file loads blogs dynamically from the backend using the Fetch API. It also handles displaying, editing, and deleting blogs.

index.js

This is the backend server file developed using Express.js. It defines all REST API routes and manages blog data.

package.json

This file stores project information and required dependencies such as Express.js.

README.md

The README file provides project documentation, installation steps, technologies used, and project overview.

10. Project Workflow

The Blog Management System follows a simple workflow to perform CRUD operations efficiently.

Step 1

The user opens the homepage using a web browser.

Step 2

The user navigates to the **Add Blog** page.

Step 3

The user enters blog details such as title, author, category, summary, and content.

Step 4

JavaScript validates the entered data to ensure all required fields are correctly filled.

Step 5

After successful validation, the frontend sends the blog data to the backend using the **Fetch API**.

Step 6

The Express.js backend receives the request and stores the blog data.

Step 7

The backend sends a success response to the frontend.

Step 8

The homepage automatically loads all available blogs and displays them to the user.

Step 9

Users can edit or delete any existing blog through the available buttons.

Step 10

The updated information is reflected immediately after the backend processes the request.

Workflow Diagram

User

|

▼

Home Page

|

▼

Add Blog Form

|

Validation

|

▼

Fetch API

|

▼

Express Server

|

CRUD Operation

|

▼

Updated Blog List

|

▼

Displayed on Home Page

11. API Details

The Blog Management System uses REST APIs to perform CRUD operations between the frontend and backend.

GET API

Endpoint

GET /blogs

Purpose

The GET API retrieves all available blog posts from the backend and displays them on the homepage.

Response

Returns a list of blog objects in JSON format.

POST API

Endpoint

POST /blogs

Purpose

The POST API creates a new blog post using the data entered in the Add Blog form.

Request Body

- Blog Title
- Author
- Email
- Category
- Summary
- Content
- Reading Time
- Difficulty Level

PUT API

Endpoint

PUT /blogs/:id

Purpose

The PUT API updates an existing blog post based on its unique ID.

DELETE API

Endpoint

DELETE /blogs/:id

Purpose

The DELETE API removes a selected blog permanently from the application.

API Communication

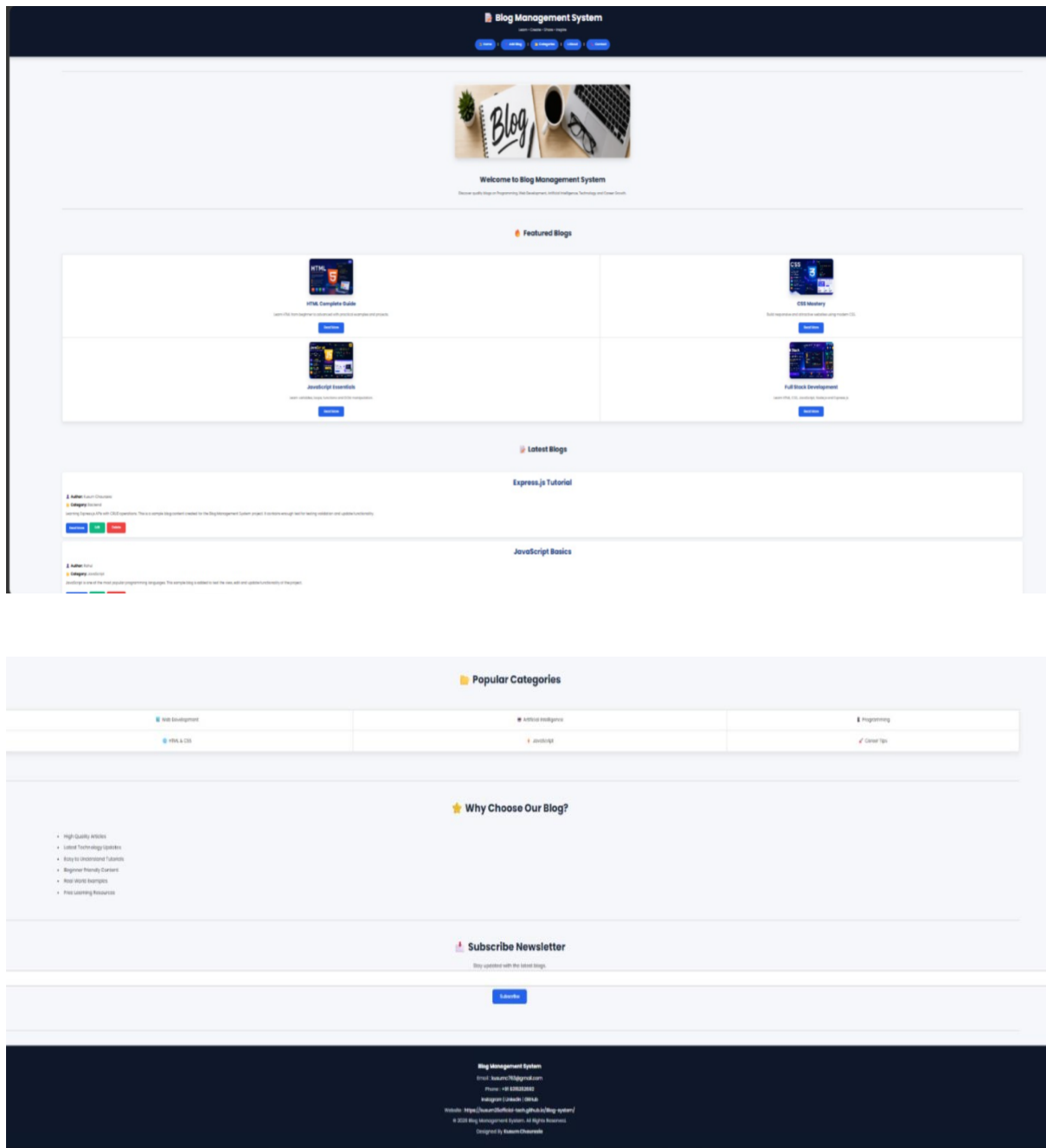
The frontend communicates with the backend using the JavaScript **Fetch API**. Whenever the user performs an action such as adding, editing, or deleting a blog, an HTTP request is sent to the Express.js server. The server processes the request and sends a JSON response back to the frontend, which updates the webpage without requiring a full page refresh.

12. Project Screenshots

The following screenshots demonstrate the implementation and functionality of the Blog Management System. These screenshots provide a clear understanding of the project interface and user experience.

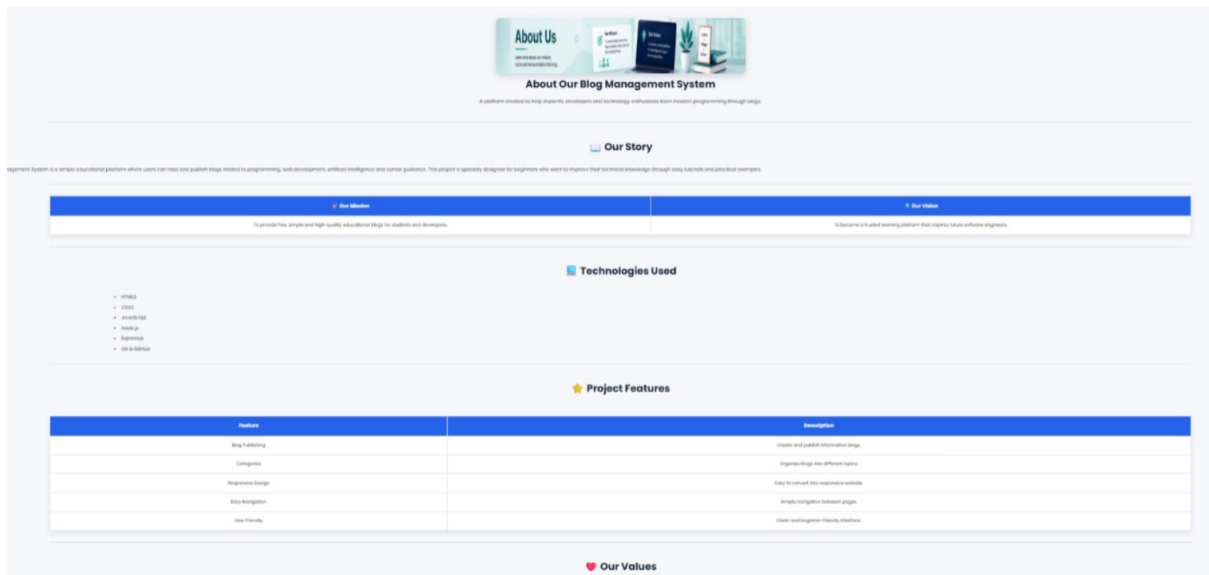
12.1 Home Page

The Home Page is the main landing page of the website. It displays the website banner, navigation bar, featured blogs, latest blogs, popular categories, newsletter subscription section, and footer. It provides users with easy navigation to all other pages of the project.



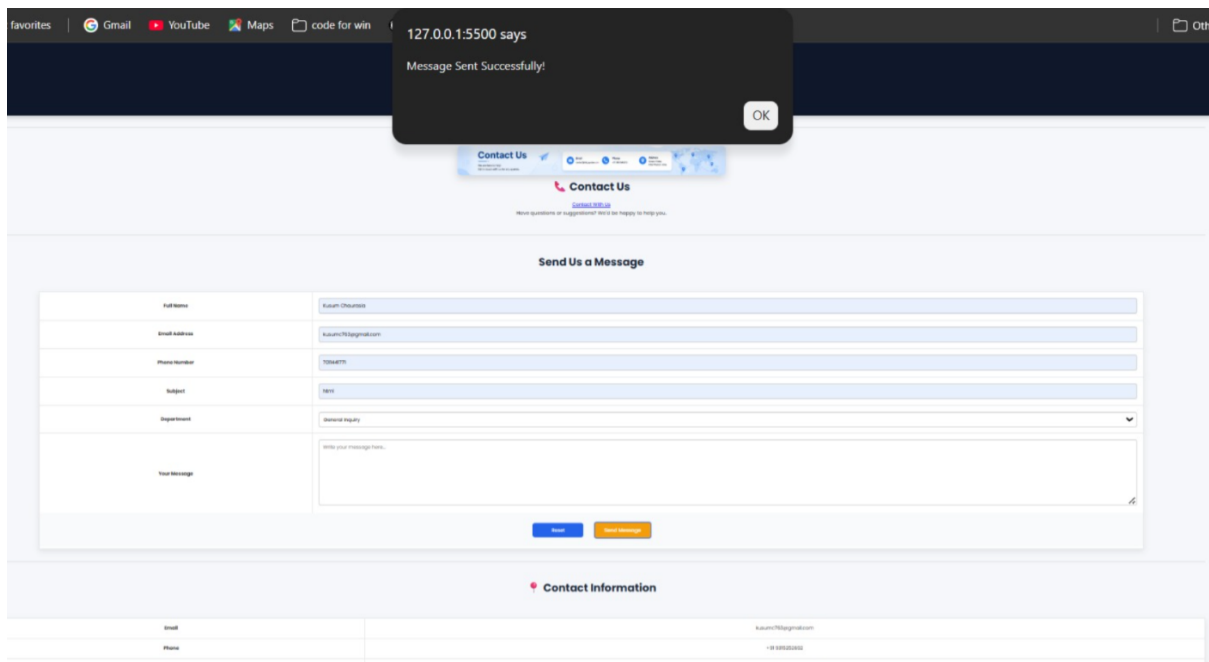
12.2 Add Blog Page

The Add Blog page allows users to create new blog posts by entering all the required information such as title, author name, category, summary, content, reading time, and difficulty level. JavaScript validation ensures that all mandatory fields are completed correctly before submission.



12.5 Contact Page

The Contact page contains a contact form, office information, working hours, social media links, and office location. Users can submit their queries using the contact form.



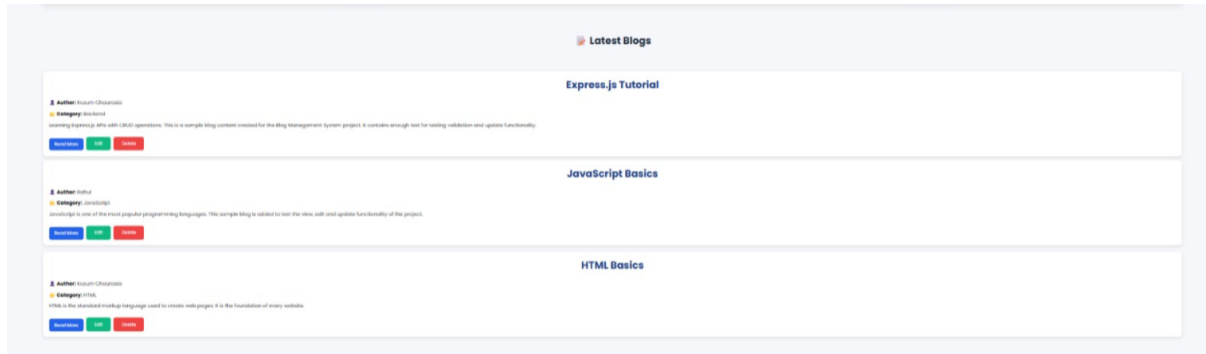
12.6 CRUD Operations

The application successfully performs the following operations:

- Add Blog

- View Blog
- Edit Blog
- Delete Blog

These operations demonstrate complete CRUD functionality implemented using Express.js REST APIs.



13. Challenges Faced

During the development of the Blog Management System, several challenges were encountered. These challenges provided valuable learning opportunities and improved my problem-solving skills.

Frontend and Backend Integration

Initially, connecting the frontend with the backend using the Fetch API was challenging. Understanding API requests and responses required careful testing and debugging.

Form Validation

Implementing JavaScript validation for different input fields required proper logic to ensure that users entered valid information before submitting the form.

REST API Development

Developing GET, POST, PUT, and DELETE APIs using Express.js required understanding routing and request handling.

Debugging

Several JavaScript errors occurred during development. Browser Developer Tools and console logs were used to identify and resolve these issues.

Git and GitHub

Managing project versions, resolving Git issues, and deploying the project using GitHub Pages helped improve version control skills.

Although these challenges required additional effort, they significantly enhanced my understanding of Full Stack Web Development.

14. Future Scope

The current version of the Blog Management System successfully performs all basic CRUD operations. However, several advanced features can be added in future versions to improve functionality.

Future improvements include:

- User Login and Registration System
- Admin Dashboard
- Database Integration using MongoDB or MySQL
- Image Upload Feature
- Search Blogs Functionality
- Blog Comments Section
- Like and Share Feature
- User Profile Management
- Email Notification System
- Dark Mode
- Blog Analytics Dashboard
- Cloud Deployment

These enhancements would make the project more scalable, secure, and suitable for real-world applications.

15. Conclusion

The **Blog Management System** project successfully demonstrates the practical implementation of Full Stack Web Development concepts. Throughout this project, I gained valuable experience in designing responsive user interfaces, developing backend APIs, integrating frontend and backend components, and managing project versions using Git and GitHub.

The project helped me understand the complete software development lifecycle, including planning, development, testing, debugging, deployment, and documentation. I also improved my skills in HTML, CSS, JavaScript, Node.js, Express.js, REST API development, and responsive web design.

Overall, this internship project enhanced my technical knowledge, problem-solving ability, and confidence in building real-world web applications. The knowledge and experience gained through this project will be valuable for future academic projects, internships, and software development careers.

16. References

The following resources were referred to during the development of this project:

- [HTML5 Official Documentation](#)
- [CSS3 Official Documentation](#)
- [JavaScript \(MDN Web Docs\)](#)
- [Node.js Official Documentation](#)
- [Express.js Official Documentation](#)
- [Git Official Documentation](#)
- [GitHub Official Documentation](#)
- [Visual Studio Code Documentation](#)
- [Postman Documentation](#)
- [Codomax Digital Solutions – Full Stack Development Internship Task Sheet.](#)